

02. DRAM 완전이해 — 정독용 심층 해설서

이 문서는 암기카드(`daily_study/A_dram.md`)와 성격이 다르다. 카드가 "외워서 즉시 꺼내는 것"이라면, 이 문서는 **앞아서 천천히 읽으며 "왜 이렇게 될 수밖에 없는가"를 1st-principles로 끝까지 따라가는 것이다**. 삼성 S.LSI에서 로직 RTL Sign-off·CDC/STA·검증을 4년 한 사람이 SK하이닉스 메모리 설계로 건너갈 때, "DRAM 디바이스"라는 낯선 땅에서 **이미 알던 디지털 설계의 직관이 어디에 닿는지를** 매 챕터 끝에서 연결한다. 면접장에서 "DRAM 설명해보세요"라는 한 마디에 30초·1분 두 버전으로 막힘없이 말하려면, 결국 이 서사 전체가 머릿속에 한 줄로 압축돼 있어야 한다.

0. 시작하기 전에 — 이 글을 읽는 자세

DRAM을 처음 공부하면 용어가 폭포처럼 쏟아진다. tRCD, tRP, tRAS, charge sharing, sense amp, refresh, bank group... 이걸 그냥 외우면 사흘이면 다 잊는다. 그런데 사실 DRAM의 거의 모든 것은 **단 하나의 가난한 사실**에서 파생된다:

"우리는 1과 0을 아주 작은 커패시터에 담긴 전하로 저장한다. 그런데 그 전하는 너무 적고, 너무 빨리 샌다."

이 한 문장이 DRAM의 운명을 결정한다. 전하가 너무 적기 때문에 → 읽을 때 신호가 미세해서 sense amp라는 정교한 증폭기가 필요하고, 읽으면 전하가 흩어져서 → 파괴적 읽기와 restore가 필연이 되고, 전하가 새기 때문에 → refresh가 필연이 되고, 더 작게 만들수록 전하가 더 줄어서 → 스케일링이 지옥이 되고, retention이 온도에 민감해서 → 고온 refresh가 두 배가 된다. 타이밍 파라미터들은 모두 "이 가난한 전하를 안전하게 다루는 데 필요한 최소 대기시간"의 다른 이름일 뿐이다.

그러니 이 글을 읽는 동안 계속 자문하라: **"지금 이 동작/제약은, 저 가난한 전하 때문에 어쩔 수 없이 생긴 것인가?"** 대부분 그렇다. 그게 보이기 시작하면 DRAM은 외울 게 거의 없는 과목이 된다.

1. 도입 — DRAM은 왜 컴퓨터의 메인메모리인가

1.1 메모리 계층이라는 큰 그림

컴퓨터의 기억장치는 하나가 아니라 **계층(hierarchy)**으로 쌓여 있다. CPU 안쪽부터 바깥으로 가면서 "빠르지만 비싸고 작은 것"에서 "느리지만 싸고 큰 것"으로 변해간다. 대략 이런 사다리다.

빠름 · 비쌈 · 작음

레지스터 (CPU 내부)	< 1ns, 수 KB	
SRAM 캐시 L1/L2/L3	1~30ns, 수 KB ~ 수십 MB	
▶ DRAM (메인메모리)	~50~100ns, 수 GB ~ 수백 GB	← 우리가 다루는 곳
NAND SSD (스토리지)	수십 μ s, 수백 GB ~ 수 TB	
HDD / 테이프	수 ms, 수 TB+	

느림 · 싼 · 큼

이 사다리에서 **DRAM**은 "**충분히 빠르면서 충분히 싸고 충분히 큰**" 절묘한 중간 지점을 차지한다. 왜 하필 DRAM이 그 자리에 앉았는지를 세 축으로 보자.

- **속도 축**: CPU는 캐시(SRAM)에서 못 찾은 데이터를 DRAM에서 가져온다. DRAM 랜덤 접근 지연은 대략 **50~100ns(추정, 동작 조건 의존)** 수준이다. SRAM보다는 한참 느리지만, NAND(수십 μ s)보다는 **수백 배 빠르다**. CPU가 매번 SSD까지 갔다 오면 연산이 거의 멈추므로, "캐시에 없는 데이터를 받아주는 빠른 풀(pool)"이 반드시 필요하고 그게 DRAM이다.
- **밀도(용량) 축**: 여기가 핵심이다. SRAM 셀은 트랜지스터 **6개(6T)**로 1비트를 만든다. DRAM 셀은 **트랜지스터 1 + 커패시터 1**, 즉 **1T1C**로 1비트를 만든다. 소자 개수만 봐도 DRAM이 훨씬 작다. 같은 다이 면적에 SRAM보다 **압도적으로 많은 비트**를 옥여넣을 수 있다. "메인메모리는 GB 단위로 커야 한다"는 요구를, SRAM으로는 면적·비용이 감당이 안 되고 DRAM만이 감당한다.
- **비용 축**: 밀도가 높다는 건 비트당 면적이 작다는 것이고, 면적이 작으면 **비트당 비용**이 싸다는 것이다. DRAM은 표준화된 대량생산(JEDEC 규격)으로 비트당 원가를 극한까지 낮춰왔다. 그래서 "큰 working memory를 싸게"라는 시스템 요구를 정확히 만족한다.

정리하면, DRAM은 "**SRAM만큼 빠르진 않지만 NAND처럼 느리지도 않고, NAND처럼 싸진 않지만 SRAM처럼 비싸지도 않은**" 자리, 즉 **속도와 밀도/비용의 균형점**에 정확히 앉아 있어서 메인메모리가 된다. 이 균형은 우연이 아니라 1T1C라는 셀 구조가 만든 필연이다.

1.2 DRAM은 "휘발성"이라는 점

DRAM은 **휘발성(volatile)**이다. 전원이 꺼지면 데이터가 사라진다. 더 정확히는 전원이 켜져 있어도 **가만 두면 사라진다**(누설). 그래서 주기적으로 되살려야(refresh) 한다. 반면 NAND는 **비휘발성**이라 전원이 없어도 유지된다. 이 차이가 "메인메모리(작업 공간) vs 스토리지(보관 창고)"라는 역할 분담의 뿌리다. 작업 공간은 빠르게 쓰고 지우는 게 중요하지 영구 보관이 목적이 아니므로, 휘발성이지만 빠르고 싼 DRAM이 적합하다.

1.3 디지털 설계자가 DRAM에서 만나는 영역 — Peri

여기서 지원자에게 가장 중요한 좌표를 미리 박아둔다. DRAM 칩은 크게 두 영역으로 나뉜다.

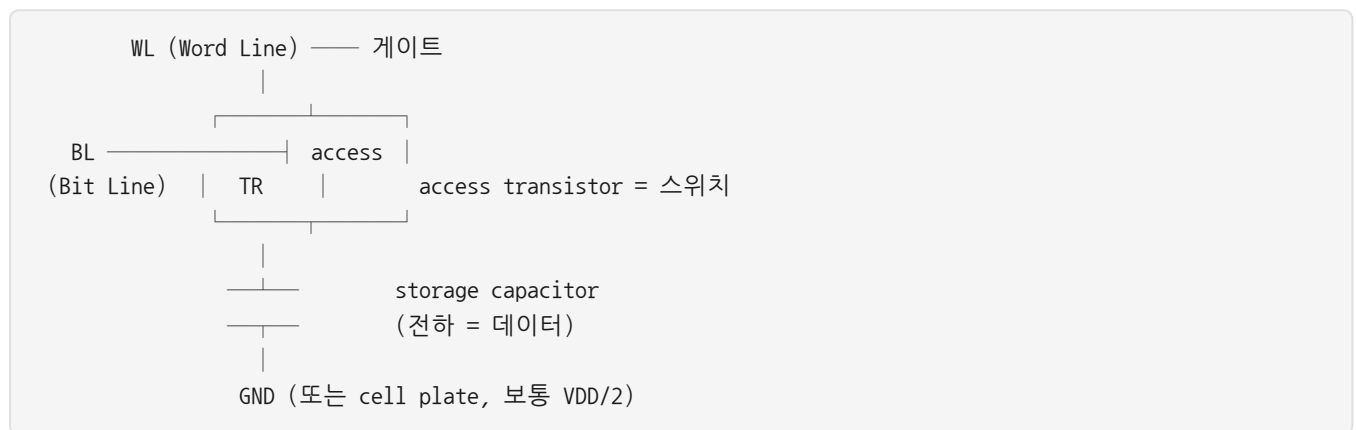
- **셀 어레이(cell array)**: 1T1C 셀이 바둑판처럼 깔린 곳. 커패시터·트랜지스터·sensing — **소자·공정·아날로그의 영역**.
- **Peri(주변회로, peripheral circuit)**: 셀 어레이를 둘러싼 모든 회로 — row/column 디코더, WL 드라이버, sense amp 제어, command/address 디코딩, refresh 컨트롤러, DLL/PLL, ECC, DQ I/O, 테스트 (MBIST·redundancy) 로직 — **디지털 RTL + 혼성신호의 영역**.

면접에서 "메모리를 잘 모르지 않냐"는 압박이 들어오면, 답의 핵심은 이것이다: **"메모리에서 RTL·타이밍·CDC·검증이 사는 곳은 정확히 Peri이고, 그건 제가 삼성에서 4년간 한 일과 같은 종류의 일입니다."** 셀은 새로 배우면 되지만, Peri는 이미 내 홈그라운드다. 이 문서가 셀(디바이스)을 깊게 파는 이유도 바로 그 셀을 "물어보면 막히는 갭"에서 "1st-principles로 설명 가능한 영역"으로 바뀌어서, 면접 전체의 자신감을 확보하기 위함이다.

2. 1T1C 셀 — 가난한 전하의 집

2.1 구조: 트랜지스터 1개 + 커패시터 1개

DRAM의 단위 셀은 딱 두 소자다.



- **access transistor(접근 트랜지스터):** 그냥 스위치라고 보면 된다. 게이트는 WL(Word Line)에 연결되고, 한쪽 단자는 BL(Bit Line)에, 다른 쪽 단자는 커패시터에 연결된다. WL에 충분한 전압을 걸면 트랜지스터가 켜져서 커패시터와 BL이 연결되고, WL을 끄면 커패시터가 BL에서 격리(isolation)되어 전하가 갇힌다.
- **storage capacitor(저장 커패시터):** 데이터 그 자체. 전하가 가득 차 있으면 1(또는 0), 비어 있으면 0(또는 1) — 어느 쪽을 1로 볼지는 설계 규약의 문제고, 본질은 "전하 있음/없음"의 두 상태다.

2.2 WL과 BL의 역할 — 행과 열

DRAM 어레이는 격자다. WL은 가로줄(row), BL은 세로줄(column)이라고 보면 된다.

- **WL(Word Line):** 한 행(row)에 속한 모든 셀의 access transistor 게이트를 동시에 제어한다. 즉 "이 행을 열어라/닫아라"의 신호다. WL 하나를 켜면 그 행에 줄줄이 매달린 수천 개 셀이 동시에 BL과 연결된다. 그래서 DRAM은 항상 **행 단위로** 활성화한다(이게 나중에 "row open"의 의미).
- **BL(Bit Line):** 한 열(column)을 따라 내려가며, 그 열에 속한 셀들이 데이터를 주고받는 통로다. 셀의 전하를 읽어낼 때도, 새 값을 써넣을 때도 BL을 통한다.

이 격자 구조에서 **하나의 셀은 (WL 하나, BL 하나)의 교차점**으로 지정된다. row 디코더가 WL을 고르고, column 디코더가 BL을 고른다 — 메모리 주소가 row 주소와 column 주소로 나뉘는 이유가 여기 있다(나중에 tRCD가 "row를 고른 뒤 column을 고르기까지의 시간"인 것과 연결된다).

2.3 왜 하필 커패시터인가 — 그리고 왜 그게 문제인가

여기서 1st-principles 질문을 던지자. "비트를 저장하는 방법은 많은데(SRAM의 래치, 플립플롭...), DRAM은 왜 굳이 커패시터를 썼나?"

답은 밀도다. SRAM처럼 래치(교차 결합 인버터)로 비트를 잡으면 안정적이지만 트랜지스터가 6개나 든다. DRAM은 "전하를 그냥 작은 통(커패시터)에 담아두자"는 가장 단순하고 작은 방식을 택했다. 소자 2개면 끝이다. 이 극단적 단순함이 곧 압도적 밀도이고, 그게 DRAM을 메인메모리로 만든 결정적 이유다.

그런데 커패시터에는 두 가지 근본적 약점이 있고, DRAM의 거의 모든 골치 아픈 특성이 이 약점에서 나온다.

(1) 저장 전하가 너무 적다 (small Q). 커패시터에 저장되는 전하량은 $Q = C \times V$ 다. 셀 커패시턴스 C는 대략 수~수십 fF(femtofarad, 10^{-15} F) 수준으로 극히 작다. V도 1V 안팎이다. 그러면 Q는 펨토쿨롱 단위의 극소량이다. 이게 왜 문제인가?

- **읽을 때 신호가 미세하다.** 나중에 보겠지만 읽기는 이 적은 전하를 긴 BL(여러 셀과 기생 커패시턴스가 줄줄이 달린 선)에 흘려보내 전압 변화를 만드는 일이다. 작은 전하를 큰 BL 커패시턴스에 나누면 전압 변화가 **고작 수십 mV**에 불과하다. 이 미세 신호를 안정적으로 1/0으로 판정하려면 **sense amp**라는 정교한 차동 증폭기가 반드시 있어야 한다. 만약 C가 너무 작으면 이 신호가 노이즈에 묻혀버려 **sensing margin(감지 여유)**이 사라지고, 오판이 난다.
- **커패시턴스가 작으면 곧 신호 마진 붕괴.** 그래서 DRAM 공정은 셀을 작게 만들면서도 C를 어떻게든 유지하려고 사투를 벌인다. 평면 면적은 줄어드는데 정전용량은 지켜야 하니, 커패시터를 위로 길게(stack) 혹은 아래로 깊게(trench) 세우고, 유전체를 high-k 물질로 바꾼다. "DRAM 미세화가 NAND보다 어렵다"는 말의 절반은 바로 이 **커패시터 정전용량 유지 난제** 때문이다(8장·9장에서 다시).

(2) 전하가 샌다 (leakage). access transistor가 "꺼져 있어도" 완벽한 절연체가 아니다. subthreshold leakage, junction leakage, 그리고 DRAM에서 특히 악명 높은 GIDL(Gate-Induced Drain Leakage) 등으로 전하가 조금씩 빠져나간다. 커패시터 자체도 유전체를 통해 미세하게 샌다. 가만 두면 수십 ms 안에 전하가 1/0 판정 경계 아래로 떨어진다. 그래서 데이터가 깨지기 전에 주기적으로 되살려야 한다 — 이것이 **refresh**의 존재 이유다(6장).

한 줄 직관: DRAM 셀은 "가장 작게(밀도) 만들 수 있는 대신, 신호가 미세하고(→ sense amp) 전하가 새는(→ refresh)" 가난한 집이다. DRAM의 모든 정교한 회로와 타이밍은 이 가난을 보살피는 장치다.

2.4 (설계 브릿지) 셀의 가난을 Peri가 보상한다

셀이 가난할수록, 그 가난을 떠받치는 **Peri의 정교함**이 중요해진다. 미세 신호를 잡아내는 sense amp 타이밍, 새는 전하를 메우는 refresh FSM, 미세화로 늘어난 불량 비트를 가리는 on-die ECC — 이 모든 보상 메커니즘이 디지털·혼성신호 로직이다. 즉 **"셀이 작아질수록 Peri의 검증 난도가 올라간다."** 삼성에서 미세화 노드의 sign-off가 점점 빡빡해지던 경험이, 여기서 그대로 "셀 가난 → Peri 정교화 → 검증 가치 상승"의 서사로 연결된다.

3. 읽기 동작 — charge sharing부터 destructive read까지

이 장이 이 문서의 심장이다. DRAM 읽기를 한 단계씩, 왜 그래야만 하는지까지 따라간다. 읽기는 크게 네 박자다: ① Precharge → ② WL on → ③ Charge sharing → ④ Sense & Restore.

3.1 ① Precharge — 왜 BL을 VDD/2로 맞추나

읽기를 시작하기 전, BL pair(BL과 BLB, 즉 비트라인과 그 짝)를 정확히 VDD/2 전압으로 미리 충전(precharge)하고 평형(equalize)시킨다. "절반 전압"이라는 게 핵심인데, 왜 0도 VDD도 아닌 정확히 한가운데인가? 세 가지 이유가 동시에 작동한다.

(1) **대칭적 신호 마진**. 셀에 저장된 값이 1이든 0이든, 읽을 때 BL은 VDD/2에서 위로(셀이 1) 또는 아래로(셀이 0) 움직인다. 출발점이 한가운데이므로 1을 읽든 0을 읽든 **신호가 움직일 여유가 양쪽으로 똑같다**. 만약 출발점을 0 근처로 잡으면 0을 읽을 때 더 내려갈 곳이 없고, VDD 근처로 잡으면 1을 읽을 때 더 올라갈 곳이 없다. VDD/2가 **최악의 경우 마진을 최대화**하는 지점이다.

(2) **차동 감지에 최적**. sense amp는 BL과 BLB 두 선의 전압 차이를 본다(차동, differential). 평형 상태에서 둘이 정확히 같은 VDD/2에 있다가, 한쪽(셀이 붙은 BL)만 살짝 움직이면, 그 미세한 차이가 곧 신호가 된다. 출발점이 정확히 같아야(완벽한 평형) 미세 차이를 깨끗하게 잡을 수 있다. 그래서 precharge에는 두 선을 같게 만드는 equalize 동작이 함께 들어간다.

(3) **전력**. BL을 VDD까지 올렸다 GND로 내렸다 하면 매번 $C \cdot VDD^2$ 의 에너지가 든다. VDD/2를 중심으로 ±절반만 흔들면 스윙이 작아 **전력이 절반 수준으로 절약**된다.

꼬리질문 대비: "왜 VDD/2 precharge?" → "1과 0 어느 쪽을 읽어도 대칭적 신호 마진을 확보하고, 차동 sense amp가 미세 차이를 최적으로 잡으며, 스윙을 줄여 전력도 아끼기 때문"이라고 세 박자로 답하면 완벽하다.

3.2 ② WL on — 행을 연다

precharge가 끝나면, 읽고 싶은 셀이 속한 행의 **WL을 활성화(on)**한다. WL 전압을 올리면(실제로는 데이터 손실 없이 1을 완전히 전달하려고 VDD보다 높은 부스트 전압 VPP까지 올린다 — 여기서 charge pump가 등장, 8장) 그 행의 access transistor가 모두 켜지고, 각 셀의 커패시터가 자신의 BL과 연결된다.

이 순간, 갇혀 있던 셀의 전하가 BL로 흘러나갈 길이 열린다. 다음 박자가 이 글 전체에서 가장 중요한 물리다.

3.3 ③ Charge sharing — 미세 신호의 탄생

WL이 켜지면 셀 커패시터(C_{cell})와 BL 기생 커패시턴스(C_{BL})가 서로 연결되어, 두 커패시터 사이에 전하가 재분배된다. 이것 **charge sharing(전하 공유)**이라 한다. 물리는 간단하다 — 연결된 두 커패시터는 전하 보존에 따라 **공통 전압**으로 수렴한다.

직관적으로 보자. 읽기 전: - BL은 precharge로 VDD/2에 있다. 거기 담긴 전하: $C_{BL} \times (VDD/2)$. - 셀은 1이면 VDD, 0이면 0V에 있었다고 하자. 담긴 전하: $C_{cell} \times V_{cell}$.

WL을 켜서 둘을 연결하면, 합쳐진 전하가 합쳐진 커패시턴스에 고루 퍼져 새 공통 전압 V_{BL} 이 된다:

$$V_{BL} = (C_{cell} \cdot V_{cell} + C_{BL} \cdot (VDD/2)) / (C_{cell} + C_{BL})$$

여기서 precharge 기준($VDD/2$)으로부터 BL이 얼마나 움직였는지(ΔV)를 보면:

$$\Delta V = V_{BL} - VDD/2 = (C_{cell} / (C_{cell} + C_{BL})) \times (V_{cell} - VDD/2)$$

이 식이 DRAM 읽기의 본질이다. 천천히 음미하자.

- 괄호 안 ($V_{cell} - VDD/2$)는 셀이 1이면 $+VDD/2$, 0이면 $-VDD/2$ 다. 즉 셀 값에 따라 ΔV 의 부호가 갈린다. 1이면 BL이 위로, 0이면 아래로 움직인다. 이 부호가 곧 우리가 읽으려는 데이터다.
- 앞의 분수 $C_{cell} / (C_{cell} + C_{BL})$ 가 신호의 크기를 결정하는 비율이다. 그런데 현실에서 C_{BL} (긴 BL에 수백 셀과 배선 기생이 붙음)이 C_{cell} (고작 수~수십 fF)보다 훨씬 크다. 비율이 보통 1/10 이하다. 그래서 ΔV 는 (작은 비율) $\times$ ($VDD/2$) = 고작 수십 mV가 된다.

이제 2장의 "가난한 전하"가 왜 문제인지 정량적으로 보인다. C_{cell} 이 작을수록(미세화로 줄수록) 위 비율이 더 작아지고, ΔV 가 더 미세해져 sensing margin이 무너진다. 반대로 C_{BL} 을 줄이려면 BL을 짧게(셀을 적게 매달아) 해야 하는데, 그러면 어레이 효율이 떨어진다. "셀 캡 : BL 캡 비율이 sensing margin을 좌우한다"는 명제가 바로 이 식의 다른 표현이다. DRAM 공정·설계가 C_{cell} 확보(high-k, 고종횡비 커패시터)와 C_{BL} 관리(BL 길이·구조, open/folded bitline)에 목숨 거는 이유가 전부 이 한 줄에 있다.

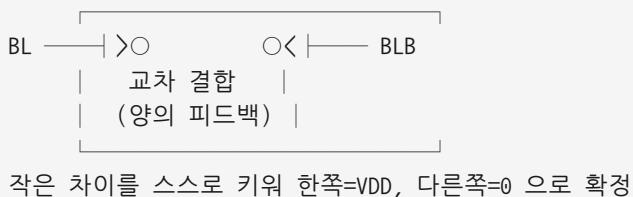
precharge	WL on, charge sharing
BL : $VDD/2$	→ $VDD/2 + \Delta V$ (셀=1, 살짝 위로)
BLB : $VDD/2$	→ $VDD/2$ (기준, 그대로)
	└─ 둘의 차이 ΔV (수십 mV) = 미세 신호

(folded bitline 구조에서는 BLB를 같은 어레이의 reference로 두어 노이즈를 공통모드로 상쇄한다 — 이 차동 구조 덕에 수십 mV의 미세 신호도 잡힌다.)

3.4 ④ Sense amp — cross-coupled latch의 차동 증폭

수십 mV의 미세한 ΔV 를 그대로 컨트롤러에 줄 수는 없다. full logic level(0 또는 VDD)로 키워야 한다. 이 일을 하는 게 sense amplifier, 보통 cross-coupled latch(교차 결합 래치) 구조다.

cross-coupled latch는 인버터 두 개를 서로의 입력과 출력에 엮갈려 묶은 것이다(SRAM 셀의 핵심과 같은 구조). 이 구조의 마법은 양의 피드백(positive feedback)이다.



작동 직관: 두 노드(BL, BLB)가 거의 같은 $VDD/2$ 근처에 있을 때, 아주 미세한 차이라도 생기면 교차 결합이 그 차이를 증폭해 한쪽을 더 끌어올리고 다른 쪽을 더 끌어내린다. 끌어올린 쪽이 다시 반대쪽을 더 끌어내리는 눈덩이

효과로, 순식간에 $BL=VDD$, $BLB=0$ (또는 그 반대)로 분리된다. 처음의 ΔV 가 +였으면 BL이 VDD로, -였으면 BL이 0으로 확정된다. 이렇게 미세 차동 신호가 full-rail 디지털 값으로 증폭된다.

sense amp의 품질이 곧 메모리의 품질이다. - **offset(오프셋)**: 두 인버터가 공정 변이로 완벽히 대칭이 아니면, 입력 신호가 0인데도 한쪽으로 기우는 경향이 생긴다. 이 offset이 ΔV 보다 크면 **오판**한다. 그래서 offset cancellation 기법이 쓰인다. - **sensing speed**: 이 증폭이 빠를수록 access time이 짧다. sensing 시간은 곧 tRCD의 물리적 구성요소다. - **noise margin**: BL noise, 인접 BL coupling이 ΔV 를 갉아먹으면 안 된다.

3.5 destructive read와 restore — 왜 다시 써야만 하는가

이제 DRAM에서 가장 반직관적이지만 가장 중요한 사실이다. **DRAM 읽기는 파괴적(destructive)이다**. 읽는 행위 자체가 데이터를 망가뜨린다.

왜 그런가? 3.3의 charge sharing을 다시 보자. 셀에 있던 전하가 BL과 공유되면서 셀 커패시터의 원래 전압이 무너졌다. 셀이 1(VDD)이었어도, charge sharing 후 셀과 BL은 같은 공통 전압($VDD/2$ 에서 ΔV 만큼 움직인 값)으로 섞여버렸다. 즉 읽는 순간 셀의 원래 전하 상태는 사라진다. 정보를 BL로 "퍼내면서" 셀 자신은 비어버린 셈이다.

그러면 데이터가 영영 사라지나? 아니다. 여기서 **restore(복원, write-back)**의 우아함이 나온다. sense amp가 ΔV 의 부호를 판정해 BL을 full-rail(VDD 또는 0)로 확정시킨 바로 그 순간, WL은 아직 켜져 있다(access transistor가 아직 열려 있다). 따라서 BL에 확정된 full 전압이 access transistor를 통해 셀 커패시터로 그대로 다시 채워진다. sense amp가 BL을 VDD로 올렸으면 셀도 VDD로 재충전되고, 0으로 내렸으면 셀도 0으로 비워진다.

즉 sense amp의 증폭 동작이 곧 셀의 복원 동작을 겸한다. 읽기 한 번에 "셀 → BL로 신호 퍼냄(파괴) → 증폭(판정) → BL → 셀로 되채움(복원)"이 자동으로 일어난다. destructive read와 restore는 분리된 두 단계가 아니라, sense amp가 한 호흡에 처리하는 동전의 양면이다.

이것이 왜 필연인가: 전하가 너무 적어(2장) 신호를 미세하게밖에 못 만드는데(3.3), 그 미세 신호를 잡으려면 셀 전하를 BL로 흘려보내(charge sharing) 셀을 비울 수밖에 없다. 비우지 않고 신호를 만드는 비파괴 방식은 더 큰 셀(더 많은 전하)을 요구하는데, 그건 밀도를 포기하는 것이다. 밀도를 위해 작은 셀을 택한 대가가 파괴적 읽기이고, 그 대가를 갚는 게 restore다. 가난한 전하 사실에서 곤장 연역된다.

이 restore가 충분히 일어나려면 시간이 든다. 그 "충분한 시간"을 보장하는 타이밍 제약이 곧 tRAS다(5장). row를 너무 빨리 닫으면(precharge하면) restore가 덜 끝나 셀 전하가 부족해지고, retention이 깨진다.

3.6 (설계 브릿지) 읽기 시퀀스 = 컨트롤러 FSM의 순서 제약

precharge → WL on → sense enable → restore → precharge로 닫기. 이 순서는 sense amp 앞단에서 보면 아날로그처럼 보이지만, 그 순서를 만들어내는 것은 메모리 컨트롤러(와 DRAM 내부 타이밍 로직)의 FSM이다. 각 단계 사이의 최소 대기시간(tRCD, tRAS, tRP...)은 디지털 설계자가 STA에서 다루는 setup/hold·multicycle path 제약과 정확히 같은 사고방식이다. "데이터가 안정되기 전에 다음 동작을 시작하면 안 된다"는 것이 setup의 본질이고, DRAM 타이밍은 그것의 프로토콜 버전이다. 삼성에서 타이밍 클로저로 보던 "안정될 때까지 기다림"을, 메모리에선 tRCD/tRAS로 본다 — 같은 문제다.

4. 쓰기 동작과 sense amp 구조 직관

4.1 쓰기는 읽기보다 "단순하지만 강제적"이다

쓰기(write)는 읽기의 거울이다. 순서는 비슷하다: precharge → WL on(행 열기) → charge sharing/sensing 시작. 그런데 여기서 한 가지가 추가된다. **column 디코더가 고른 BL에 컨트롤러가 외부에서 강제로 쓸 값(0 또는 VDD)을 인가한다.** sense amp가 약한 셀 신호로 한쪽을 정하려는 그 순간, 외부 write 드라이버가 더 강하게 BL을 원하는 값으로 밀어버린다. 그러면 sense amp의 latch도 그 값으로 뒤집혀 확정되고, WL이 아직 열려 있으니 그 값이 셀로 써진다.

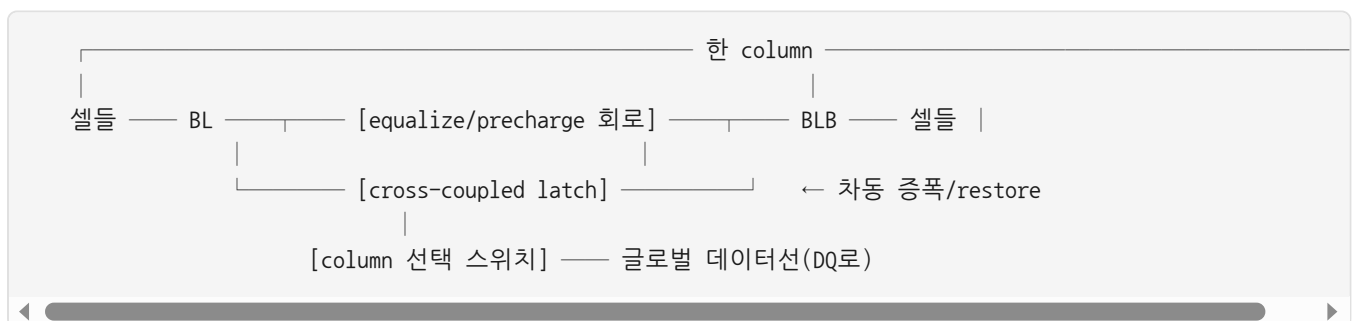
요점은 두 가지다. - 쓰기는 결국 "원하는 값으로 sense amp latch를 강제 세팅"하는 일이다. 그래서 쓰기에서도 sense amp가 중심에 있다. - 한 행을 활성화하면 그 행 전체가 sense amp에 올라온다(open). 그 중 일부 column만 새 값으로 덮어쓰고, 나머지 column은 읽힌 그대로 restore된다. 즉 부분 쓰기를 해도 **나머지 셀들의 restore는 자동으로 함께 일어난다.** 이게 DRAM이 항상 "행 단위로 열고(activate), 행 단위로 닫는(precharge)" 이유와 맞물린다.

4.2 tWR — 쓰기가 셀에 "다 스며들" 시간

쓰기에서 중요한 타이밍이 tWR(Write Recovery)다. write 드라이버가 마지막 데이터를 BL에 밀어넣은 뒤, 그 값이 access transistor를 통해 셀 커패시터에 충분히 채워질 때까지 기다려야 한다. 이 시간을 안 기다리고 곧장 precharge로 행을 닫으면(WL off), 셀이 덜 채워진 상태로 격리되어 쓴 값이 약하게 들어가 retention이 위태로워진다. 그래서 "마지막 write 데이터 → precharge"까지의 최소 시간이 tWR로 강제된다. 본질은 tRAS와 같다 — "전하를 셀에 충분히 넣을 시간을 벌어주는 것."

4.3 sense amp 구조 직관 정리

sense amp를 한 그림으로 정리하면:

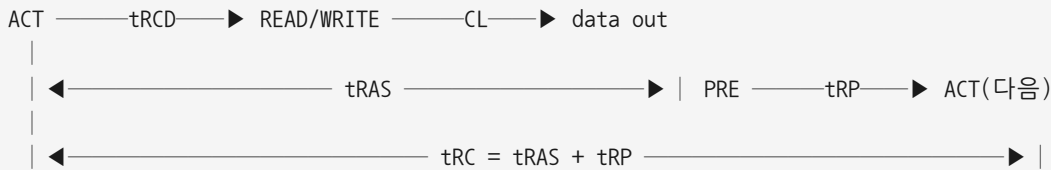


- **precharge/equalize 회로**: 읽기 전 BL/BLB를 VDD/2로 같게 만든다.
- **cross-coupled latch**: 미세 ΔV 를 full-rail로 증폭 + 셀 restore.
- **column 선택**: 활성화된 행의 수천 column 중, 지금 읽거나 쓸 column만 글로벌 데이터선에 연결해 DQ I/O로 보낸다(읽기) 또는 외부 값을 받는다(쓰기).

이 구조를 머릿속에 가지고 있으면 "sense amp 동작 설명해보세요"에 그림 그리듯 답할 수 있다.

5. 타이밍 파라미터 완전이해 — 각각이 "무엇을 기다리는 시간"인가

DRAM 타이밍 파라미터는 외울 대상이 아니라 "무슨 물리가 끝나길 기다리는가"로 이해할 대상이다. 모두 같은 **bank 기준**(같은 행을 다루는 동안의 시퀀스)이라는 점을 기억하자. 동작의 흐름과 함께 보면 자연스럽다.



5.1 tRCD (RAS-to-CAS Delay) — 행을 열고, 열이 준비될 때까지

ACTIVATE(행 열기, RAS) 명령을 내린 뒤 → READ/WRITE(열 선택, CAS) 명령을 받을 수 있을 때까지의 최소 시간. 물리적으로 이 시간 동안 일어나는 일: WL이 켜지고(VPP까지 부스트), charge sharing이 일어나고, sense amp가 ΔV 를 잡아 BL을 안정시킨다. 즉 "행을 열어 데이터가 sense amp에 안정적으로 올라올 때까지"의 시간이다. 이게 끝나야 비로소 그 행의 특정 column을 읽거나 쓸 수 있다. - 너무 짧으면? sense amp가 아직 증폭을 못 끝낸 상태에서 column을 읽으니, 불안정한 중간 전압을 읽어 오판한다.

5.2 CL = tCAS (CAS Latency) — 읽기 명령 후 데이터가 나오기까지

READ 명령을 내린 뒤 → 첫 데이터가 DQ 핀에 나오기까지의 시간, 보통 클럭 사이클 수로 표기한다(예: CL=40). column을 골라 글로벌 데이터선·I/O 경로를 거쳐 외부로 내보내는 데 드는 시간이다. - random read latency $\approx$ tRCD + CL: 임의의 주소를 읽으려면 먼저 행을 열고(tRCD) 그 다음 열을 읽어 데이터를 받는다(CL). 그래서 "닫혀 있던 행에서 처음 데이터를 받기까지"의 총 지연이 대략 tRCD + CL이다. (행이 이미 열려 있으면(row hit) tRCD를 건너뛰어 CL만으로 빠르게 읽는다 — 이게 row buffer locality, bank가 latency를 숨기는 메커니즘과 연결.)

5.3 tRP (Row Precharge) — 행을 닫고 다음 행을 열 준비

PRECHARGE 명령을 내린 뒤 → 다음 ACTIVATE를 할 수 있을 때까지의 최소 시간. 이 시간 동안 WL을 끄고(행 닫기), BL을 다시 VDD/2로 precharge·equalize해서 다음 읽기를 위한 깨끗한 출발점을 만든다. - 너무 짧으면? BL이 아직 VDD/2로 평형되지 않았는데 다음 행을 여니, 출발점이 한쪽으로 기울어 charge sharing의 미세 신호가 오염되고 오판이 난다(3.1의 대칭 마진이 깨짐).

5.4 tRAS (Row Active) — 행을 열어둬야 하는 최소 시간

ACTIVATE 후 → PRECHARGE까지의 최소 시간. 이건 "최대"가 아니라 "최소"라는 게 핵심이다. 행을 너무 빨리 닫지 못하게 막는 하한이다. 왜? 3.5에서 본 restore 때문이다. sense amp가 셀을 다시 채워넣는 데 시간이 걸리는데, restore가 끝나기 전에 precharge로 WL을 꺼버리면 셀이 덜 채워진 채 격리되어 데이터가 약해진다. tRAS는 "한 번 연 행은 적어도 이만큼은 열어둬서 restore를 끝내라"는 보장이다. - 너무 짧으면? restore 미완 → 셀 전하 부족 → retention 악화, 데이터 깨짐. 그래서 tRAS에는 물리적 하한이 있다.

5.5 tRC (Row Cycle) — 한 행을 열고 닫는 전체 주기

tRC = tRAS + tRP. ACTIVATE에서 다음 ACTIVATE까지, 즉 한 행을 완전히 열고(활성·restore: tRAS) 다시 닫는(precharge: tRP) 전체 사이클이다. 같은 bank에서 다른 행을 연달아 칠 때(row conflict) 이 tRC가

throughput을 옥죈다. 그래서 컨트롤러는 같은 bank의 다른 행을 연속으로 때리는 패턴을 피하고, 여러 bank로 분산해 tRC를 숨기려 한다(7장).

5.6 tWR (Write Recovery) — 쓴 값이 셀에 정착할 시간

4.2에서 본 대로, 마지막 write 데이터 → PRECHARGE까지의 최소 시간. 셀에 새 값이 충분히 충전될 시간을 보장한다. 너무 짧으면 약하게 써진 셀이 retention을 못 버틴다.

5.7 (설계 브릿지) 타이밍 파라미터 = 메모리판 STA 제약 집합

이 파라미터들은 메모리 컨트롤러가 지키는 타이밍 제약의 집합이고, 여기면 functional fail이다. 디지털 설계에서 setup 위반이 "데이터가 안정되기 전에 클럭이 잡는 것"이듯, tRCD 위반은 "sense amp가 안정되기 전에 column을 읽는 것"이다. tRAS 하한은 "restore가 끝날 시간을 보장하는 multicycle 제약" 같은 것이다. 삼성에서 PrimeTime으로 보던 "min/max delay, 안정 윈도우" 사고가 그대로 DRAM 프로토콜 타이밍으로 옮겨온다. 메모리 컨트롤러 설계·검증에서 이 제약들을 RTL FSM과 STA로 보장하는 일이 곧 내 일이다.

6. Refresh와 Retention — 새는 전하와의 끝없는 싸움

6.1 retention time — 셀이 데이터를 버티는 시간

2.3에서 봤듯 셀 전하는 누설로 샌다. 셀이 데이터를 판정 가능한 수준으로 유지하는 시간을 retention time이라 한다. JEDEC 표준은 상온에서 64ms, 그리고 85°C를 넘는 고온에서는 절반인 32ms로 본다. 이 "64ms"라는 숫자가 refresh 주기 계산의 출발점이다.

왜 고온에서 절반인가? 누설(특히 subthreshold leakage, junction leakage)은 온도에 지수적으로 민감하다. 온도가 오르면 캐리어가 더 활발해져 누설이 급증하고, 전하가 더 빨리 샌다. 그래서 retention이 짧아지고, refresh를 더 자주 해야 한다(temperature-compensated refresh). 데이터센터·AI 서버처럼 뜨거운 환경에서 refresh 오버헤드가 커지는 현실적 이유가 여기 있다.

6.2 refresh — 주기적 read + rewrite

retention이 끝나기 전에 데이터를 되살려야 한다. 방법은 의외로 단순하다 — 그냥 그 행을 한 번 activate해서 읽으면, restore(3.5) 과정이 자동으로 셀을 다시 채운다. 즉 refresh는 본질적으로 "읽어서 다시 써넣기(read + rewrite)"이고, 그 rewrite는 이미 있는 restore 메커니즘을 재활용한다. 외부로 데이터를 내보낼 필요 없이, 행을 열었다 닫기만 하면 그 행의 모든 셀이 새로 충전된다.

6.3 tREFI $\approx$ 7.8 μ s — 7.8마이크로초의 산수

이제 핵심 숫자 하나를 직접 유도하자. 한 DRAM 뱅크/디바이스에 행이 예컨대 8192개라 하고, 모든 행을 64ms 안에 한 번씩 refresh해야 한다면, 평균적으로 얼마마다 refresh 명령을 발행해야 할까?

$$t_{REFI} \approx 64\text{ms} / 8192 \text{ 행} \approx 7.8\mu\text{s}$$

즉 평균 7.8 μ s마다 한 번씩 auto-refresh를 발행하면 64ms 안에 8192행을 모두 돌 수 있다. 이 tREFI(Refresh Interval)가 DRAM 동작의 배경 리듬이다. (실제로는 한 refresh 명령이 여러 행을 묶어 처리하거나, 고온에선

간격을 절반($3.9\mu\text{s}$)으로 줄이는 등 변형이 있다. 행 개수도 용량·구성에 따라 다르다 — 8192는 전형적인 예시 값.)

- **auto-refresh**: 정상 동작 중, 컨트롤러가 tREFI마다 REFRESH 명령을 발행하면 DRAM이 내부 카운터로 "다음 차례 행"을 알아서 refresh한다.
- **self-refresh**: 시스템이 저전력 standby로 들어가 컨트롤러가 쉴 때, **DRAM 칩 스스로** 내부 타이머·온도 센서로 refresh를 돌린다(저전력 모드). LPDDR은 여기에 deep power-down, **PASR(Partial Array Self-Refresh, 안 쓰는 영역은 refresh 생략)** 같은 추가 절전 기법을 엮는다.

6.4 refresh의 대가와 완화

refresh는 공짜가 아니다. - **대역폭 손실**: refresh 중인 bank는 정상 read/write를 못 받는다. 용량이 클수록(행이 많을수록), 고온일수록(주기가 짧을수록) 이 손실이 커진다. - **전력**: 매 $7.8\mu\text{s}$ 마다 행을 열고 닫으니 가만 있어도 전력이 든다.

DDR5는 이를 완화하려 **fine-granularity refresh**(전체를 잘게 쪼개 한 번에 적게), **same-bank refresh**(특정 bank만 refresh하고 나머지는 계속 접근 가능) 등을 도입했다.

6.5 미세화가 retention을 악화시키는 이유

DRAM 스케일링의 근본 난제가 바로 retention이다. 셀을 작게 만들면: - **저장 전하 Q가 준다**(C가 작아짐, 2.3). 같은 양의 누설이라도 **전체 전하 대비 비중이 커져** 더 빨리 판정 경계 아래로 떨어진다. - access transistor가 작아지며 **GIDL 등 누설이 오히려 늘기 쉽다**(short channel effect). - 셀 간격이 좁아져 **인접 셀 간섭(coupling)**과 **RowHammer**(특정 행 반복 접근이 인접 행 전하를 흔드는 현상) 같은 신뢰성 문제가 심해진다.

즉 "작게 만들면 더 빨리 새고, 더 자주 refresh해야 하고, 신호 마진도 줄어든다." 그래서 미세 노드일수록 retention 분포 관리, on-die ECC(미세 불량 보정), TRR(RowHammer 완화) 같은 보강이 필수가 된다. retention은 DRAM 미세화의 벽 그 자체다.

6.6 (면접 핵심) "DRAM은 왜 refresh가 필요한가" — 완성형 논리

이 질문은 ★기출 빈출이다. 1st-principles 논리 사슬로 답하면 흔들림이 없다:

"DRAM 셀은 전하를 작은 커패시터에 저장하는데, access transistor 누설·접합 누설·GIDL로 전하가 수십 ms 안에 쏩니다(retention 표준 64ms, 고온 32ms). 데이터가 깨지기 전에 주기적으로 행을 읽어 restore로 다시 채워야 하는데 이것이 refresh이고, 8192행 정도를 64ms 안에 돌려면 평균 $7.8\mu\text{s}$ 마다(tREFI) auto-refresh를 발행합니다. 미세화로 전하가 줄고 누설이 늘수록 retention이 악화돼 refresh 부담이 커지는 것이 DRAM 스케일링의 근본 난제입니다. 고온일수록 누설이 지수적으로 늘어 retention이 절반으로 줄고요."

꼬리질문 대비: self-refresh vs auto-refresh, refresh의 대역폭·전력 오버헤드와 완화(per-bank/fine-granularity), RowHammer와 TRR의 관계까지 한 발씩 더 준비.

7. Bank와 Bank Group — 병렬성으로 지연을 숨기다

7.1 문제의식: 한 행을 열고 닫는 건 느리다

5장에서 봤듯 한 행을 열고(tRCD) 닫는(tRP) 전체 사이클 tRC는 길다. 만약 DRAM이 거대한 단일 어레이라면, 한 번에 한 행만 다룰 수 있어서 매 접근마다 이 긴 지연을 고스란히 떠안아야 한다. throughput이 처참할 것이다. 해결책은 나눠서 동시에 일하기다.

7.2 Bank — 독립적으로 행을 여는 단위

DRAM을 여러 개의 독립 어레이인 bank로 나눈다. 각 bank는 자기만의 sense amp 행(row buffer)을 갖고, 독립적으로 행 하나를 open할 수 있다. 그러면 마법이 일어난다:

- bank 0에서 행을 열고(activate, tRCD 대기 중)...
- 그 대기 시간 동안 bank 1을 접근하고...
- bank 0이 준비되면 데이터를 받고, 동시에 bank 1을 닫고...

이렇게 여러 bank의 동작을 시간축에서 겹쳐(overlap) 한 bank의 긴 지연을 다른 bank의 작업으로 숨긴다(hide latency). 디지털 설계의 파이프라이닝/리소스 분할과 똑같은 사고다 — 자원을 나눠 동시성을 만들어 throughput을 끌어올린다.



7.3 Bank Group — DDR4/5의 한 겹 더

DDR4부터는 bank를 다시 bank group(BG)으로 묶었다. 왜 또 묶나? 속도가 너무 빨라지면서, 같은 BG 안의 회로(공유 데이터 경로 등)는 연속 접근 시 회복할 시간이 필요해졌기 때문이다. 여기서 두 타이밍이 갈린다:

- tCCD_L (CAS-to-CAS Delay, Long): 같은 BG 내 연속 column 접근의 최소 간격 — 느리다(공유 회로 회복 대기).
- tCCD_S (Short): 다른 BG로 연속 접근의 최소 간격 — 빠르다(서로 독립이라 바로 가능).

그래서 컨트롤러는 연속 요청을 일부러 서로 다른 BG로 분산시킨다. 그러면 tCCD_S(빠른 간격)로 burst를 촘촘히 이어붙여, 데이터 버스를 빈틈없이 채울 수 있다. 같은 BG만 연속으로 때리면 tCCD_L에 걸려 버스에 구멍이 생긴다. "BG를 분산하라"는 컨트롤러 스케줄링의 황금률이 여기서 나온다.

7.4 DDR5의 32 bank

DDR5 16Gb 디바이스는 전형적으로 $8 \text{ BG} \times 4 \text{ bank} = 32 \text{ bank}$ 구성이다. DDR4(보통 16 bank)보다 bank를 두 배로 늘려, 더 많은 독립 행을 동시에 운용하고 더 깊은 병렬성으로 고속 데이터 레이트(4800~8800 MT/s)를 뒷받침한다.

- bank를 무한정 늘리면? 각 bank마다 디코더·sense amp 등 peri 오버헤드와 면적이 늘고, 제어 복잡도가 커진다. 그래서 trade-off가 있고, 무작정 늘리지 않는다.

7.5 (설계 브릿지) bank 병렬성 = 자원 분할로 만든 동시성

bank/BG는 자원을 쪼개 동시성을 만들어 throughput을 올리는 전형적 아키텍처 패턴이다. 디지털 설계의 파이프라인 스테이지 분할, 멀티포트/멀티뱅크 SRAM, 병렬 처리 유닛과 같은 직관이다. 그리고 이 모든 분산·overlap을 언제 어느 bank/BG를 때릴지 결정하는 것은 메모리 컨트롤러의 arbitration·scheduling 로직 — 전형적인 디지털 제어 블록이고, 정확히 내 검증·STA·CDC가 닿는 영역이다.

8. Peri(주변회로) — DRAM 안의 디지털 설계 영역

이 장은 자원자에게 이 문서 전체에서 가장 전략적인 챕터다. 셀(소자)은 새로 배워야 할 갭이지만, Peri는 이미 내 홈그라운드라는 프레임을 확실히 세운다.

8.1 Peri를 구성하는 블록들

cell array를 제외한 모든 회로가 Peri다. 읽기/쓰기/refresh 한 사이클을 따라가며 Peri를 만나보자.

- **command/address 디코딩**: 외부에서 들어온 명령(ACT/READ/WRITE/PRE/REF)과 주소를 해석한다. 전형적 디지털 디코더·FSM.
- **row decoder**: 주소를 받아 수천 WL 중 하나를 고른다. (column decoder도 마찬가지로 BL/column 선택.)
- **WL driver**: 고른 WL을 켜다. 이때 1을 손실 없이 전달하려고 VDD보다 높은 부스트 전압 VPP가 필요하다.
- **charge pump(차지 펌프)**: 외부엔 VDD밖에 없는데 VPP(WL 부스트)·VBB(기판 바이어스) 같은 VDD 밖의 전압을 칩 내부에서 만들어내는 회로. 커패시터 스위칭으로 전압을 끌어올린다. 아날로그·혼성신호지만 그 제어 타이밍은 디지털이다.
- **sense amp + 제어**: 3·4장의 그 sense amp. 어레이에 붙어 있지만 언제 enable할지(precharge → WL → sense enable 순서)는 디지털 타이밍 제어가 결정한다.
- **refresh controller**: tREFI마다 refresh를 발행하고 다음 행 카운터를 관리하는 백그라운드 FSM. 정상 read/write와 충돌 안 나게 스케줄링.
- **DLL/PLL**: 외부 I/O 클럭과 내부 클럭을 정렬한다. 특히 고속 DDR/HBM I/O에서 데이터-스트로브(DQ-DQS) 타이밍을 맞추는 핵심.
- **ECC 엔진(on-die ECC)**: 미세 노드에서 늘어난 single-bit 셀 오류를 칩 내부에서 정정. 인코더/디코더는 전형적 RTL.
- **DFT/MBIST/redundancy/repair**: 칩 내부 자가 테스트 엔진(March 패턴 등), 불량 셀을 여분 row/column으로 대체(BIRA/BISR), eFuse. 수율 직결의 디지털 로직.
- **DQ I/O**: 외부와 데이터를 주고받는 고속 인터페이스(드라이버, 등화 DFE 등).

8.2 "Peri = 메모리 안의 디지털 설계 영역"이라는 핵심 프레임

위 목록을 보라. row/column decoder, command FSM, refresh controller, ECC 인/디코더, DLL/PLL, MBIST, redundancy 제어 — 이것들은 전부 RTL로 짜고, STA로 타이밍 달고, CDC로 클럭 도메인 검증하고, LEC로 정합성 확인하고, UVM/SVA로 검증하는 디지털 블록이다. 셀 어레이가 "소자·아날로그"라면, Peri는 정확히 "디지털 RTL + 혼성신호"이고, 이게 메모리에서 RTL·타이밍·CDC가 사는 곳이다.

특히 Peri는 다중 클럭 도메인(코어 / I/O / DLL·PLL / refresh)이 공존하므로 CDC가 본질적으로 핵심이고, 다중 reset 도메인(power-on / soft / mode)이 있어 RDC도 그대로 적용된다. DDR/HBM I/O의 빠빠한 setup/hold는

STA sign-off 그 자체다. 삼성에서 한 일이 좌표만 바뀌어 그대로 살아난다.

8.3 COP — Cell-over-Peri

미세화의 압박은 셀만 받는 게 아니다. 셀은 $4F^2 \cdot 3D$ 로 줄어드는데(9장), Peri 로직과 sense amp는 그만큼 줄지 않는다. 그래서 칩 면적에서 Peri 비중이 점점 커진다. 이를 완화하려고 Peri를 셀 어레이 아래로 넣는 Cell-over-Peri(COP) 구조를 쓴다(셀을 peri 위에 적층). 같은 면적에 더 많은 셀을 올려 어레이 효율을 높인다.

8.4 (면접 핵심 브릿지) "메모리를 몰라서가 아니라, Peri가 곧 내가 하던 일"

면접에서 "메모리 디바이스 경험이 없지 않냐"는 질문이 들어오면, Peri 프레임으로 정면 돌파한다:

"메모리에서 RTL·타이밍·CDC·검증이 사는 곳은 셀 어레이가 아니라 Peri입니다. row/column decoder, refresh controller, ECC, DLL/PLL, MBIST, redundancy — 모두 다중 클럭·다중 reset 도메인을 가진 디지털 블록이고, 제가 삼성에서 4년간 CDC/RDC sign-off, STA timing closure, LEC 정합성, 검증 자동화로 다룬 것과 같은 종류의 문제입니다. 셀(디바이스)은 새로 배우면 되지만, Peri는 제 홈그라운드입니다. 그리고 미세화로 셀이 작아질수록 on-die ECC·정교한 sense amp 타이밍·TRR처럼 Peri의 검증 난도가 올라가므로, 제 sign-off·자동화 경험의 가치가 오히려 커지는 영역이라고 봅니다."

9. 스케일링 — $6F^2$ 에서 $4F^2$, 그리고 3D DRAM의 벽

9.1 F와 셀 면적 — $6F^2$ 의 의미

F는 공정의 최소 피쳐 크기(minimum feature size)다. 셀 면적을 nF^2 로 표기하는 건 "최소 피쳐의 제곱(F^2) 단위로 몇 배 면적을 차지하나"를 뜻한다. 오랫동안 평면 DRAM 셀의 표준은 $6F^2$ (보통 $2F \times 3F$)였다. WL·BL 배선과 access transistor·커패시터·격리를 평면에 배치하면 $6F^2$ 정도가 한계였다.

9.2 $4F^2$ — 트랜지스터를 세우다 (VCT / Vertical Gate)

면적을 더 줄려면 평면 배치로는 한계다. 그래서 access transistor를 수직으로 세운다. 채널을 수직 방향으로 만드는 VCT(Vertical Channel Transistor) + Vertical Gate(VG) 구조로 가면, 셀을 $4F^2(2F \times 2F)$ 까지 줄일 수 있다. 같은 공정 노드에서 다이 면적을 약 30% 줄이는 효과다(SK하이닉스 VLSI 2025 로드맵 발표). 전류 경로가 "커패시터 → 수직 채널 → 매립 BL"로 짧아지는 부수 이점도 있다.

평면 $6F^2$ ($2F \times 3F$)

수직 $4F^2$ ($2F \times 2F$)

TR이 옆으로 누움
배선이 면적을 먹음

TR을 위로 세움(VCT)
같은 노드, 다이 ~30% ↓

9.3 노드 1a / 1b / 1c — "10nm-class"의 세대 표기

DRAM은 더 이상 "XXnm"로 딱 떨어지게 부르지 않고 10nm-class를 1a/1b/1c...로 세대 표기한다. 대략 1a ≈ 14, 1b ≈ 12, 1c ≈ 11nm급으로 통한다(정확한 치수는 비공개·정의가 회사마다 다름 — 통상적 대응 관계, 추정).

1c는 2025~26년 양산 단계로, EUV를 5~6 레이어 적용하며, 1c DDR5는 약 8Gbps급(전세대 대비 +14% 속도 / -9% 전력, 발표치)로 알려졌다. 미세화 한 세대마다 retention·sensing margin·peri 면적 압박이 모두 커지므로, 노드 진전은 곧 검증·sign-off 수요 증가를 뜻한다.

9.4 EUV의 역할

미세 패턴을 그리려면 노광 파장이 짧아야 한다. EUV(극자외선, 13.5nm 파장)는 기존 ArF(193nm) 대비 훨씬 미세한 패턴을 적은 마스크 단계로 그릴 수 있어, 1a 후반·1b·1c 노드의 핵심 공정이 됐다. EUV 레이어 수가 늘수록 미세화는 가능해지지만 비용·난도도 오른다.

9.5 3D DRAM은 왜 NAND처럼 쉽지 않은가

NAND는 이미 셀을 수백 단 수직 적층(V-NAND)한다. "DRAM도 그렇게 쌓으면 되지 않나?"가 자연스러운 질문이고, 실제로 3D DRAM이 차세대 방향으로 연구된다. 그런데 DRAM의 3D 적층은 NAND보다 훨씬 어렵다. 이유를 1st-principles로 보자.

- NAND 셀은 전하를 트랩(charge trap)에 가두는 단순 구조라, 수직 채널 홀을 뚫고 거기에 셀을 줄줄이 만드는 적층이 상대적으로 수월하다(그래도 고종횡비 식각은 극한 기술).
- DRAM 셀은 "충분한 정전용량의 커패시터"가 필수다(2.3, 3.3). sensing margin과 retention이 모두 C에 달려 있기 때문이다. 그런데 셀을 작게·여러 층으로 쌓으면서도 각 셀마다 충분한 커패시턴스를 확보하려면, 커패시터를 극도로 높은 종횡비(길쭉하게)로 만들어야 한다. 이 고종횡비 커패시터를 수직 적층 구조에서 균일하게·신뢰성 있게 제조하는 게 현재 기술의 벽이다. 게다가 access transistor와 커패시터를 함께 3D로 쌓아야 하니 NAND보다 구조가 복잡하다.

꼬리질문 대비: "DRAM이 NAND처럼 쉽게 3D로 못 간 이유?" → "DRAM은 sensing/retention을 위해 셀마다 충분한 커패시턴스를 유지해야 하는데, 면적을 줄이며 그 정전용량을 확보하려면 커패시터를 고종횡비로 만들어야 하고, 그걸 수직 적층에서 균일하게 제조하는 난도가 NAND의 charge-trap 적층보다 훨씬 높기 때문"이라고 답한다.

9.6 (설계 브릿지) 구조 전환기일수록 검증 수요가 폭증한다

4F²·3D처럼 셀이 작아지고 구조가 바뀔수록 신호 마진이 줄어, on-die ECC·정교한 sense amp 타이밍·새로운 redundancy/repair가 더 중요해진다. 즉 소자 미세화가 곧 회로·검증 난도 상승이다. 새 구조 전환기는 새 검증 방법론·새 sign-off 수요가 가장 큰 시기이고, 그게 EDA 자동화·검증 강점을 가진 지원자에게는 기회다.

10. 제품군 — 같은 셀, 다른 전략 (DDR / LPDDR / GDDR / HBM)

DRAM의 셀(1T1C)은 같다. 달라지는 건 타깃 시장과 I/O 전략이다. "I/O를 어떻게 굴릴 것인가"가 제품군을 가른다.

10.1 네 가지 제품의 성격

- DDR (Double Data Rate): 서버·PC용 범용 메모리. DIMM 모듈 형태로 메인보드에 꽂는다. 용량·속도·전력의 균형형(DDR5 4800~8800 MT/s). "메인메모리" 하면 보통 이것.

- LPDDR (Low Power DDR): 모바일·노트북용 저전력 특화. 저전압, deep power-down·PASR 같은 절전 모드, package-on-package(point-to-point) 실장으로 전력·면적을 최소화. 배터리 수명이 곧 가치인 시장.
- GDDR (Graphics DDR): GPU용. per-pin 속도를 극단으로(GDDR6 16~24Gbps급) 끌어올린다. 핀 하나하나를 초고속으로 굴러 그래픽 대역폭을 확보하는 대신 전력이 크다. I/O 폭보다 per-pin 속도로 승부.
- HBM (High Bandwidth Memory): DRAM die를 TSV(Through-Silicon Via)로 수직 적층(8~16Hi)하고 base die(logic) 위에 올려, 1024-bit(HBM3E) / 2048-bit(HBM4)의 초광폭 I/O를 중간 속도로 굴린다. 2.5D 인터포저로 GPU와 연결. per-pin 속도를 무리하게 올리지 않고 폭(width)으로 총대역폭을 확보 → bit당 에너지 최저 + 총대역폭 최고 + 면적 최소.

10.2 I/O 전략의 trade-off — 폭 vs per-pin 속도

여기에 디지털 설계자가 바로 알아볼 SerDes·버스 폭 설계의 trade-off가 있다.

- per-pin 속도를 올리면(GDDR): 핀 수는 적게 유지하지만, 고속에서 ISI(심볼간 간섭)·SI 문제가 심해져 DFE 같은 등화(equalization)가 필요하고 전력이 크다.
- I/O 폭을 늘리면(HBM): per-pin 속도는 낮춰 SI는 상대적으로 수월하지만, 수천 개 핀(TSV)이 필요해 적층·인터포저·패키징 난도가 폭증한다.

HBM은 후자를 택해, 같은 대역폭을 훨씬 낮은 전력·작은 면적으로 달성한다. AI 가속기가 GDDR 대신 HBM을 쓰는 이유가 이것이다 — TB/s급 대역폭을 합리적 전력·면적에 집적할 수 있는 유일한 길이기 때문. (HBM 자체의 적층·TSV·MR-MUF·열·수율 병목은 면접 은행 A8 참고; 이 문서는 DRAM 디바이스 본질에 집중.)

10.3 DDR5 신기능 — on-die ECC / DFE / sub-channel

DDR5는 고속·미세화 시대의 요구를 반영한 신기능을 담았다.

- on-die ECC: 미세 노드에서 늘어나는 single-bit 셀 불량률 칩 내부에서 보정해 수율·신뢰성을 지킨다. 주의 — on-die ECC는 셀 단위 보정이고, 시스템 ECC(rank 단위, 링크·소자 전체 오류 검출/정정)는 역할이 다르다. on-die ECC가 있다고 시스템 ECC가 불필요해지지 않는다.
- DFE (Decision Feedback Equalizer, 보통 4-tap): 고속(8000+ MT/s)에서 심해지는 ISI를 receiver에서 보상해 eye margin을 회복한다.
- sub-channel: DIMM을 독립 32-bit sub-channel 2개로 쪼개 효율을 높인다(기존 64-bit 1채널 → 32-bit 2채널). 짧은 burst도 효율적으로 처리. on-DIMM PMIC로 전원을 모듈에서 직접 관리.
- 속도: 4800 → 8800 MT/s(JESD79-5C, 2024.04 기준)로 확장.

10.4 (설계 브릿지) I/O 전략 = SerDes·SI·등화의 디지털/혼성 설계

폭 vs per-pin 속도는 SerDes·버스 폭 설계와 동형이고, DFE·on-die ECC는 SI·신뢰성 회로다. 디지털 설계자가 STA에서 보는 jitter/eye margin, ECC 인/디코더 RTL과 직접 연결된다. 즉 제품군의 차이는 곧 I/O·검증 전략의 차이이고, 그 전략을 RTL·STA·CDC로 구현·검증하는 게 내 일이다.

11. 면접 연결 — 구술 골격과 브릿지

이 장은 앞 내용을 입으로 말하는 형태로 압축한다. ★는 빈출 기출이다. 각 질문에 30초(핵심만)와 1분(논리 사슬) 두 버전을 준비한다.

11.1 ★ "DRAM을 설명해보세요"

[30초] "DRAM은 1T1C, 즉 트랜지스터 1개와 커패시터 1개로 1비트를 저장하는 휘발성 메인메모리입니다. 커패시터 전하로 1/0을 담고, WL로 행을 열어 BL과 charge sharing시킨 미세 신호를 sense amp가 증폭해 읽습니다. 읽기가 파괴적이라 restore가 동반되고, 전하가 새기 때문에 주기적 refresh가 필요합니다. 밀도·속도·비용의 균형이 좋아 메인메모리가 됐습니다."

[1분] 위에 더해: "셀이 작아서 저장 전하가 적고, 그래서 읽을 때 BL에 만들어지는 신호가 수십 mV로 미세합니다. 이걸 VDD/2 precharge 후 charge sharing으로 만들어 cross-coupled latch sense amp가 차동 증폭합니다. 읽으면 셀 전하가 BL로 흩어져 파괴되므로, sense amp가 BL을 full-rail로 확정하는 그 동작으로 셀을 다시 채우는 restore가 함께 일어납니다. 누설로 전하가 64ms(고온 32ms) 안에 새기 때문에 평균 7.8μs마다 refresh로 되살립니다. 타이밍은 tRCD(행 열고 sense 안정)·tRAS(restore 보장)·tRP(precharge)·CL(데이터 출력)로 규정되고, random read latency는 대략 tRCD+CL입니다. 여러 bank로 나눠 지연을 숨기고, 디지털 로직은 전부 Peri에 있습니다 — 거기가 제 RTL·CDC·STA 경험이 닿는 곳입니다."

11.2 ★ "DRAM은 왜 refresh가 필요한가요?"

[30초] "셀 커패시터 전하가 누설로 시간이 지나면 사라지기 때문입니다. 데이터가 깨지기 전에 행을 읽어 restore로 다시 채워야 하고, 이게 refresh입니다. retention 표준 64ms(고온 32ms) 안에 모든 행을 돌려야 해서 평균 7.8μs마다 auto-refresh를 발행합니다."

[1분] 6.6의 완성형 논리(누설 종류 → retention → restore 재활용 → tREFI 산수 → 미세화·온도 악화 → 완화 기법)를 그대로. 꼬리: self-refresh(저전력 standby에서 칩 스스로) vs auto-refresh, refresh의 대역폭·전력 오버헤드와 same-bank/fine-granularity 완화, RowHammer-TRR 연결.

11.3 ★ "DRAM vs NAND의 근본적 차이는?"

[30초·4축] "휘발성(DRAM 휘발성 / NAND 비휘발성), 셀 구조(DRAM 1T1C 커패시터 / NAND charge-trap·floating-gate, NAND string 직렬), 액세스 단위(DRAM 랜덤 byte/word / NAND 페이지 read·블록 erase), 용도·특성(DRAM 빠르고 refresh 필요한 메인메모리 / NAND 느리지만 비휘발·P/E 마모 있는 스토리지). 한마디로 DRAM은 작업 공간, NAND는 보관 창고입니다."

[1분] 위에 "왜 그런가"를 덧붙임: "DRAM은 커패시터 전하라 빠르게 쓰고 지우지만 새기 때문에 refresh가 필요하고 휘발성입니다. NAND는 전하를 절연된 트랩에 가둬 비휘발이지만, 쓰려면 터널링으로 oxide를 통과시켜야 해 느리고 P/E 반복으로 oxide가 마모돼 수명이 제한됩니다. 그래서 NAND는 3D 적층이 비교적 수월했지만(charge trap), DRAM은 sensing·retention용 커패시터 확보 때문에 3D가 훨씬 어렵습니다. 다만 둘 다 셀 바깥의 Peri·컨트롤러 디지털 로직은 공통이고, 거기가 제 검증·sign-off가 닿는 영역입니다."

11.4 "sense amp 동작을 설명해보세요"

[30초] "BL pair를 VDD/2로 precharge하고, WL을 켜면 셀과 BL의 charge sharing으로 수십 mV 차이가 생깁니다. cross-coupled latch가 양의 피드백으로 이 미세 차동 신호를 full-rail(0/VDD)로 증폭하고, 동시에 WL이 켜져 있으니 확정된 값이 셀로 다시 써지는 restore가 일어납니다."

[1분] 3.3~3.5의 charge sharing 식($\Delta V = C_{\text{cell}} / (C_{\text{cell}} + C_{\text{BL}}) \times (V_{\text{cell}} - VDD/2)$)과 부호=데이터, 비율=신호크기, offset/margin, restore의 필연성까지. 꼬리: VDD/2 precharge 이유(대칭 마진+차동 최적+전력), offset cancellation, cell cap : BL cap 비율과 sensing margin.

11.5 삼성 로직 → 메모리 Peri 브릿지 — 한 문장 라이브러리

각 답변 끝에 붙일 "내가 하던 X = 메모리의 Y" 한 문장들:

- **CDC/RDC**: "사업부 CDC sign-off와 RDC false violation 87.7% 감소 방법론을 했는데, 메모리 Peri는 코어/I/O/DLL·PLL/refresh의 다중 클럭·다중 reset 도메인이라 CDC/RDC가 그대로 핵심입니다."
- **STA**: "PrimeTime으로 timing closure를 직접 했는데, DDR/HBM I/O의 뽀뽀한 setup/hold 마진이 정확히 같은 STA 문제입니다. DRAM 타이밍(tRCD/tRAS/tRP)도 결국 setup/hold·multicycle 사고와 동형입니다."
- **LEC**: "CPU IP hardening에서 약 1,760만 line 규모 EQ 이중화를 했는데, 메모리 Peri 합성·ECO 정합성 검증에 그대로 이식됩니다."
- **AI-EDA**: "CDC 분류 ML→LLM PoC, CSR→constraint·assertion 자동생성을 했는데, 미세화로 Peri 검증 난도가 오를수록 이 자동화 가치가 커집니다."
- **종합 한 문장**: "메모리를 몰라서가 아니라, Peri가 곧 제가 하던 일입니다. 셀(디바이스)은 배우면 되고, Peri는 제 홈그라운드입니다."

12. 스스로 점검 — 이 질문에 막힘없이 답할 수 있는가

아래 10개에 1st-principles로(외운 게 아니라 "왜"로) 답할 수 있으면 이 문서를 소화한 것이다. 막히는 항목은 해당 장으로 돌아가라.

1. DRAM이 메모리 계층에서 "메인메모리" 자리에 앉은 이유를 **속도·밀도·비용** 세 축으로 설명할 수 있는가? (1장)
2. DRAM의 거의 모든 특성(sense amp / restore / refresh / 스케일링 난제)이 "**가난한 전하**"라는 한 사실에서 어떻게 연역되는지 한 줄로 꿰어 말할 수 있는가? (0·2장)
3. **왜 BL을 VDD/2로 precharge**하는지 세 가지 이유(대칭 마진·차동 최적·전력)를 댈 수 있는가? (3.1)
4. charge sharing 식 $\Delta V = C_{\text{cell}} / (C_{\text{cell}} + C_{\text{BL}}) \times (V_{\text{cell}} - V_{\text{DD}}/2)$ 에서, 왜 신호가 수십 mV로 미세하고 왜 미세화가 sensing margin을 무너뜨리는지 설명할 수 있는가? (3.3)
5. DRAM 읽기가 **왜 파괴적이며**, restore가 **왜 sense amp 동작과 한 호흡인지** 말할 수 있는가? (3.5)
6. tRCD / tRP / tRAS / tRC / CL / tWR 각각이 "무슨 물리가 끝나길 기다리는 시간"인지, 너무 짧으면 무슨 일이 나는지 댈 수 있는가? 그리고 tRC=tRAS+tRP, latency≈tRCD+CL을 유도할 수 있는가? (5장)
7. tREFI ≈ 7.8μs를 retention 64ms와 행 수(예 8192)로부터 직접 산수로 유도할 수 있는가? 고온에서 32ms로 주는 이유(온도-누설 관계)는? (6장)
8. bank와 bank group이 어떻게 latency를 숨기는지, tCCD_L/tCCD_S가 왜 갈리고 컨트롤러가 왜 BG를 분산하는지 설명할 수 있는가? (7장)
9. Peri가 "**DRAM 안의 디지털 설계 영역**"임을 구체 블록(decoder/refresh controller/ECC/DLL/MBIST...)으로 들고, 내 CDC/STA/검증 경험과 연결할 수 있는가? COP는? (8장)
10. DRAM이 NAND처럼 3D로 가기 어려운 이유(커패시턴스 확보·고종횡비 커패시터)를 1st-principles로 말할 수 있는가? $6F^2 \rightarrow 4F^2$ (VCT/VG), 노드 1a/1b/1c, EUV는? (9장)

보너스 — 면접 직전 자문: ① ★기출(DRAM 설명 / refresh 왜 / DRAM vs NAND / sense amp)을 30초·1분 두 버전으로 말할 수 있는가? ② 모든 답 끝에 Peri 브릿지 한 문장을 붙일 수 있는가? ③ "메모리 디바이스를 모르지 않냐"는 압박에 "모릅니다"가 아니라 "셀은 배우면 되고 Peri는 제 홈그라운드"로 받아칠 수 있는가?

달는 말: DRAM은 외울 게 많아 보이지만, "작은 커패시터에 담긴 가난한 전하"라는 한 사실이 모든 것을 낳는다. 신호가 미세해서 sense amp, 읽으면 깨져서 restore, 새어서 refresh, 더 줄이면 더 새서 스케일링 난제. 이 인과의 사슬을 손에 쥐면, 어떤 꼬리질문도 "그건 이 사실에서 이렇게 나옵니다"로 되돌릴 수 있다. 그리고 그 모든 보상 메커니즘이 사는 곳 — Peri — 이 바로 당신이 4년간 살던 동네다.